Node.js під капотом: runtime, процеси та масштабування

Заняття №1

Node.js Pro

Лектор
Кирило Орлов

Кирило Орлов

Tech Lead та Principal Software Engineer

Лектор курсу

- за плечима — понад 30 сервісів у продакшн на multi-tenant архітектурі
-
- 35 реалізованих проєктів і майже 30 000 годин практики в програмуванні
-
- це вже третій потік курсу Node.js PRO — курс уже пройшли понад 100 випускників

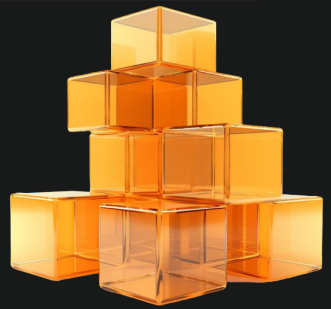


МАРШРУТ КУРСУ: 34 ЗАНЯТТЯ

33 лекції + захист курсового · вт і чт 18:30 · фундамент → Docker → NestJS зсередини → дані → комунікації → Redis + security → production (K8s, AWS, CI/CD)

Принципи, а не заклинання: на кожну технологію — альтернативи і decision matrix (Kafka vs RabbitMQ, cluster vs worker_threads)

Production scars: реальні кейси з продакшну з цифрами. Outcome курсу — власний задеплоєний production-ready сервіс



БАЛИ: 200 ЗА КУРС

157

22 ДЗ: 2 розігрівні (#3 HTTP-сервер на net, #5 Docker) → mini-Nest трилогія (#6–8: власний міні-NestJS, 23 бали) → 17 проєктних ДЗ

23

11 тестів у LMS: 5–10 хв після лекції, 1–3 бали. Перший — сьогодні

20

Захист курсового проєкту на занятті #34: production backend, зібраний з 17 проєктних ДЗ

HW0

Без балів, перед #9: architecture decision record курсового — фундамент для всіх проєктних ДЗ

ПРАВИЛА ГРИ

Формат: recap 3 хв → теорія → демо → міст до наступної. Питання — в чат по ходу, Q&A наприкінці

Перед лекцією — prep-brief у LMS (10–15 хв). Запис ефіру і всі матеріали — в LMS

v2 = перебудований після brutal-розбору v1. Я відкритий до критики: не працює методика — пиши в чат/анкету, міняю по ходу курсу



ПЛАН ЗАНЯТТЯ



- Recap: стек Node.js та проблема C10K
- Event Loop: фази, nextTick, thread pool
- V8: компіляція, hidden classes, inline caches
- Garbage Collector: heap, алгоритми, memory leaks
- Process model: Cluster vs Worker Threads
- Підсумки: міфи та міст до лекції 2

ВСТУП

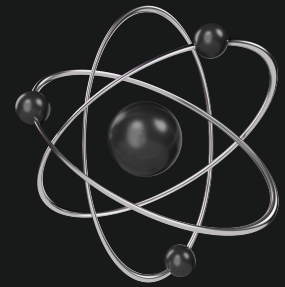


СТЕК NODE.JS СЬОГОДНІ

Node.js runtime = 4 шари: твій код → Stdlib (JS) → C++ bindings → V8 + libuv

Масштабування: cluster (CPU) + worker_threads (CPU-bound) + PM2 (production)

Сьогодні пройдемо всі 6 компонентів згори донизу — і повернемось до цієї карти кілька разів



4 ФАКТИ-СЮРПРИЗИ ПРО NODE.JS

4 РІВНІ	×4 ПОТОКИ	100 000 TCP	≥1 МС
JS ніколи не виконується в машинному коді одразу — проходить 4 рівні компіляції	fs.readFile() за замовчуванням — це 4 потоки в пулі	Один Node.js процес тримає 100 000 TCP-з'єднань на одному потоці	setTimeout(fn, 0) НЕ виконується через 0 мс — мінімум 1 мс

ПРОБЛЕМА C10K

THREAD-PER-REQUEST

Apache: 1 з'єднання = 1 потік, ~1-2 MB RAM на стек. 10 000 з'єднань = 20 GB RAM тільки на стеки

EVENT-DRIVEN (NODE.JS)

Рішення Райана Дала (2009): один потік, event-driven I/O, ніколи не чекає синхронно. 10 000 з'єднань ≈ 100 MB RAM

C10K PROBLEM

Dan Kegel, 1999 — формалізував проблему: тисячі одночасних з'єднань ламають thread-per-request модель

До 2009: серверні мови (Java/PHP/Python) = thread-per-request

АНАЛОГІЯ: РЕСТОРАН ⇔ NODE.JS

V8 = шеф-кухар (виконує рецепти-код) · libuv = менеджер залу (координує чергу замовлень)

C++ bindings = вікно видачі між кухнею та залом · Node.js Stdlib = меню для клієнта

Event loop = графік роботи менеджера · Thread pool = бригада прибиральниць (фонова робота)



АРХІТЕКТУРА: 4 ШАРИ + ЗАПУСК ПРОЦЕСУ

Шари: твій код (app.js) → lib/ чистий JS → src/ C++ bindings → deps/v8 + deps/uv

Запуск node app.js: процес ОС → V8 Isolate → libuv loop → built-in модулі → компіляція app.js → event loop крутиться

Isolate = ізольований V8 instance; Worker Thread = другий Isolate в тому ж процесі



APACHE VS NGINX VS NODE.JS

APACHE

Thread-per-request; 10K з'єднань = 10K потоків; ~20 GB RAM

NGINX

Event-driven (C); 10K з'єднань = 4-8 потоків; ~50 MB RAM

NODE.JS

Event-driven (JS): 1 JS thread + 4 thread pool; ~100 MB RAM; чому не Go/Rust? — онбординг, npm (2M+ пакетів)

V8 ENGINE



V8 PIPELINE: 4 РІВНІ КОМПІЛЯЦІЇ

Ignition:

bytecode interpreter після parsing (lexer → AST, lazy parsing); збирає type feedback

Sparkplug:

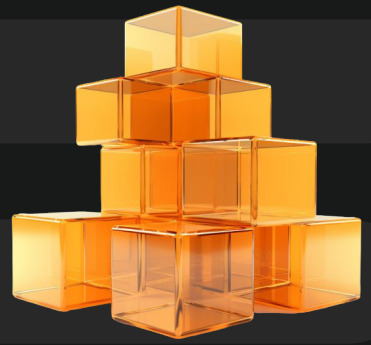
baseline compiler, неоптимізований машинний код, ~5-15% швидше Ignition

Maglev:

CFG-based, type specialization, ~10× швидша компіляція за TurboFan

TurboFan:

sea-of-nodes IR, inlining, escape analysis — «compile only what's hot»



4 РІВНІ ГОТОВКИ (АНАЛОГІЯ ДО V8 TIERS)

Ignition

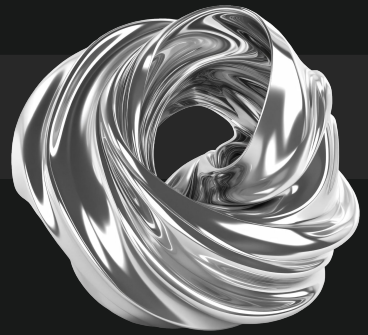
читаєш рецепт по тексту прямо під час готування

Sparkplug

вивчив рецепт напам'ять — готуєш не дивлячись у книгу

Maglev → TurboFan

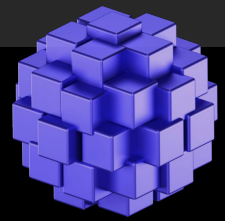
додав улюблені shortcuts → усе оптимізовано заздалегідь: духовка розігріта, інгредієнти готові



HIDDEN CLASSES + INLINE CACHES

Hidden Class = «креслення будинку»: однаковий порядок ініціалізації → один shape → прямий offset-доступ

Inline Cache: monomorphic (найшвидше) → polymorphic (2-4 shapes) → megamorphic (5+, до 100× повільніше)



```
function Point(x, y) {
  this.x = x;  // C1: {x at 0}
  this.y = y;  // C2: {x:0, y:1}
}
const p1 = new Point(1, 2);  // → C2
const p2 = new Point(3, 4);  // → C2 (same shape!)

// BAD — різний порядок → різні shapes
const a = { x: 1 }; a.y = 2;  // {x,y}
const b = { y: 2 }; b.x = 1;  // {y,x} DIFFERENT!
// Production scar: webhooks з 8 SaaS → megamorphic IC, p99 80ms → нормалізація payload → p99 8ms
```

GARBAGE COLLECTOR

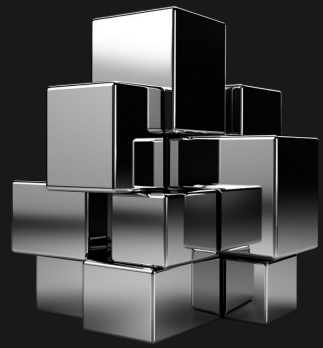


GC: HEAP LAYOUT + GENERATIONAL HYPOTHESIS

V8 heap = набір spaces: New Space (1-8 MB), Old Space (~1.5 GB), Large Object, Code, Map

Generational hypothesis (1980-ті): більшість об'єктів помирають молодими

Young gen чистимо часто і швидко; Old gen — рідко і повільно



SCAVENGE + MARK-SWEEP-COMPACT + ORINOCO

Scavenge (Young):

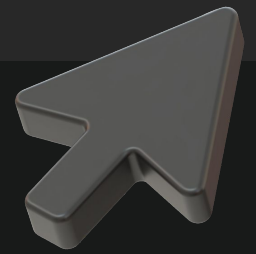
два semi-spaces From/To, bump-pointer allocation; копіюються лише живі (10-20%); 2 виживання → промоція в Old

Mark-Sweep-Compact:

tri-color marking (white/grey/black) → sweeping → compacting для Old Gen

Orinoco:

incremental + concurrent marking/sweeping, idle-time GC; було stop-the-world до 1 с → тепер паузи < 10 мс



MEMORY LEAKS: ТИПИ + ІНСТРУМЕНТИ

Причини: глобальні змінні, closures з великим scope, listeners без off(), setInterval без clear, unbounded caches

Production scar: cache ролей без TTL → RSS 4 GB за 6 год → kubelet killed pod; fix — lru-cache з maxAge

Інструменти: --inspect + Heap Snapshot, process.memoryUsage(), --trace-gc, --max-old-space-size



EVENT LOOP



EVENT LOOP: 6 ФАЗ

Timers → Pending callbacks → Idle/Prepare → Poll (серце event loop) → Check → Close callbacks

Poll: рахує час блокування, виконує I/O callbacks; Check: setImmediate — завжди після poll

Між КОЖНОЮ фазою: повний дренаж process.nextTick queue, потім microtask queue



QUIZ: NEXTTICK VS MICROTASK VS SETIMMEDIATE

```
const fs = require('fs');
console.log("1");
setTimeout(() => console.log("2"), 0);
fs.readFile(__filename, () => {
  console.log("3");
  setTimeout(() => console.log("4"), 0);
  setImmediate(() => console.log("5"));
  process.nextTick(() => console.log("6"));
  Promise.resolve().then(() => console.log("7"));
});
setImmediate(() => console.log("8"));
Promise.resolve().then(() => console.log("9"));
process.nextTick(() => console.log("10"));
console.log("11");
// Result: 1 11 10 9 2 8 3 6 7 5 4
```



LIBUV THREAD POOL + OS I/O

THREAD POOL (4 потоки): fs.*, dns.lookup(), crypto (pbkdf2/randomBytes/scrypt), zlib — 5-й запит чекає в черзі. UV_THREADPOOL_SIZE=16 — до 1024. Gotcha: sync pbkdf2 + 100 RPS → starvation, p99 → секунди

NETWORK I/O — НЕ thread pool: epoll (Linux) / kqueue (macOS) / IOCP (Windows) — до 100K TCP-з'єднань на одному JS потоці



MODULE SYSTEM



MODULE SYSTEM: REQUIRE() + CIRCULAR DEPS

3 типи specifier: core module → file path → bare specifier (пошук по node_modules)

```
// a.js
const b = require('./b');
exports.foo = "A's foo";
exports.fromB = b.bar;

// b.js — circular!
const a = require('./a');
exports.bar = "B's bar";
exports.fromA = a.foo; // undefined
```

Module cache: require.cache, ключ = повний шлях; circular deps → partial exports, не crash



PROCESS MODEL



HTTP-сервер, що масштабується на CPU → Cluster (PM2 -i max) — isolation, port sharing через SO_REUSEPORT

CPU-heavy задача у HTTP-запиті → Worker Threads — не блокує event loop

Pre-fork pool / microservice scaling → child_process.spawn / K8s — менший overhead, ізоляція

CLUSTER VS WORKER THREADS: DECISION MATRIX

ПІДСУМКИ + Q&A



8 МІФІВ ПРО NODE.JS

- «V8 компілює весь JS» — лише hot paths досягають TurboFan
- «Node.js однопоточний» — libuv pool / GC / OS I/O багатопоточні
- «GC паузи довгі» — Orinoco: < 10 мс
- «Circular requires крешають» — повертають partial exports, не crash
- «Event loop — проста черга» — 6 фаз з microtask drain між кожним callback
- «setImmediate швидший за setTimeout(0)» — недетерміновано поза I/O callback
- «Workers ділять об'єкти як Java» — окремий isolate, лише SharedArrayBuffer
- «nextTick = microtask» — окрема, вища черга



МІСТ ДО ЛЕКЦІЇ 2: ASYNC PATTERNS + STREAMS

Сьогодні: архітектура + внутрішнє виконання — V8, GC, event loop, process model

Node.js добре тримає I/O-навантаження, поки код не блокує event loop синхронним CPU-heavy виконанням

Лекція #2: Promises deep dive, AsyncLocalStorage, AbortController; Streams (Readable/Writable/Transform) і backpressure — як не з'їсти 10 GB RAM при стрімінгу файлу

Pre-read до #2: концепт [[backpressure]] в Obsidian vault (3 хв)

Q&A

???



YOUR PLACE
TO GROW